

the Perl programming community and later adopted by other programming language communities. But Perl's reputation as the best programming language for code golfing has been challenged in recent years.

Many programmers became so immersed in code golfing puzzles, some of them even developed programming languages that are suitable for code golfing challenges alone. One such option is the aptly named *GolfScript*, an esoteric programming language solely developed to win such competitions. But do remember that programs written in such programming languages are not at all readable and, after a few years, even the person who wrote the program will have a hard time understanding it. But, as mentioned earlier, the only requirement for code golfing competitions is to write programs with the least source code and nothing more.

## A quick look at code obfuscation

Now let us discuss code obfuscation, yet another recreational programming activity. Code obfuscation involves writing programs that are difficult for human beings to understand. To illustrate this point further, consider the program *ob1.c* given below:

```
#include <stdio.h>
#include <string.h>
#define _____ ;
#define What p##r##i##n##t##f(
#define is "%s\n"
#define This ,____);}
#define _____ m##a#####i##n
#define __o__ int
#define __0__ char
#define __0__ strcpy
#define ooo ++
#define oo /2
__o__ _____(_____, __0__
*____){__o__
_=0;__0__ _____[32]_____ __0__ _____
[32] _____ __0__
(_____, "aHbeghl1oj oWuotrk1mdq!b"
) _____
```

```
for (____=0 _____ <26 _____ _ ooo )
_____[_ oo ]
=____[++] _____
What
is
This
```

I had written this program many years ago for my blog. Can you guess the output of the program? It prints the message 'Hello World!' Though the program looks like a mess, there is nothing clever in it. It uses the C preprocessor to define alternative symbols for many commonly occurring elements in C. The program also uses the fact that variable names in C can begin with an underscore symbol. Thus, many of the lines you see in the program are actually predefined macro variables. Similarly, if you observe carefully, the letters in the even positions of the string *aHbeghl1oj oWuotrk1mdq!b* form the string *Hello World!*. The program also uses the token pasting operator (*##*) of the C preprocessor to combine multiple tokens into a single token. For example, *m##a#####i##n* is the same as *main*. Figure 2 shows the output of the program *ob1.c*, which prints the message 'Hello World!'.

Figure 2: Output of the program *ob1.c*

Now consider the program *ob2.c* given below. This is another program I had written for my blog a long time back.

```
#include<stdio.h>
#define _ ;
#define _ ,
#define _ (
#define _ )
#define _ {
#define _ }
#define _ +
#define _ printf
```

```
#define _____ scanf
#define _____ while
#define _____ int
#define _____ main
#define 00 =
#define 000 1
#define o "
#define ooo "%d"
#define oooo <=
#define oooo *
#define ooo &
#define ooooo [
#define ooooo ]
#define o0 char
o0 __0__ ooooo 20 ooooo 00 _____ 69 __
78 __ 84 __ 69 __ 82 __
32 __ 78 __ 79 __ 58 _____
o0 __00__ ooooo 30 ooooo 00 _____ 70 __
65 __ 67 __ 84 __
79 __ 82 __ 73 __ 65 __ 76 __ 32 __ 73
__ 83 __ 58 _____
_____
_____ 0 00 000 __ 0000 __
00000 00 000 __
_____ __0__ _____
_____ __ooo__ o0o 0000 _____
_____ 0 oooo 0000 _____
_____
00000 00 00000 oooo 0 _
0 00 0 _____ 000 _
_____
_____ __00__ _____
_____ __ooo__ 00000 _____
```

Again, it is nothing to be proud of nor is it too clever; it is just about abusing the C preprocessor and variable naming conventions and, maybe, required a bit of patience. Now can you guess what the program is doing? Figure 3 shows the output of the program *ob2.c*, which reads a number from the keyboard and finds its factorial. Notice that the command *gcc -w ob2.c* is used to compile the program *ob2.c*, so that all the warnings are ignored. Further, notice that the strings *ENTER NO:* and *FACTORIAL IS:* are printed using the numeric ASCII value of the characters. For example, the ASCII value 69 corresponds to the letter *E*.